

CS310 Natural Language Processing
Assignment 4: SFT for LLM
Total points: 30

Goal: Run supervised fine-tuning (SFT) for a GPT-2 model (355M parameters) on a simple instruction following dataset (`instruction-data.json`, ~1k samples).

Submit:

- The complete training script `run_sft.py`.
- The generated model responses, e.g., `responses.json`.
- Write up a PDF report to include the results for Requirement 3. (see highlights)

Requirements:

1) (10 points) Complete the code for running the SFT task.

- The weights of a pretrained GPT-2 model is provided (`gpt2-355M.pth`). You need to first instantiate a `GPTModel` object, and then call the `load_state_dict` method, together with `torch.load` to load the pretrained weights.
- Reuse the code from the lab, including the `format_input` function, the `InstructionDataset` class, and the custom collate function `custom_collate_fn`.
- Implement the `init_data_loaders` function, which includes the code for datasets creation and data loaders initialization.
- Implement the `train_model` function, in which the main training loop exists. It is in similar structure as the `train_model_simple` function used in the pretraining assignment, but takes fewer input arguments. It should return the lists of training and validation losses that will be used for plotting their curves.

2) (10 points) Masking out the instruction and input part.

- A new class `InstructionDatasetMask` is to be implemented. It contains a new member `self.instruction_lengths`, which stores the lengths of the concatenated instruction and input strings. It marks the tokens that need be replaced with `ignore_index=-100`.
- Correspondingly, a new function `custom_collate_fn_mask` needs be implemented. There is only one major difference from `custom_collate_fn`, which is to replace the tokens in instruction and input with `-100`.
- A “`--mask_instructions`” argument is prepared to control which way of processing to execute.
- After all training epochs are done, remember to use `model.state_dict` and `torch.save` to save the fine-tuned model weights, e.g., `sft_model.pth`.

3) (10 points) Generate responses on the test data and evaluate them using LLM-as-a-judge.

- Implement the `generate` function, which takes the fine-tuned model and the token IDs of the instruction and input, and generate certain number of new tokens. No random sampling is needed, that is, a simple greedy decoding strategy is all you need. Also, KV-cache is optional, as efficiency is not our main consideration here.
- You can call the `generate` function to the end of training in the `main` function. You can write a standalone script for convenient use on any fine-tuned model. A “`--generate_response`” argument is prepared to control this action. When this option is turned on, you should save the generated response in an JSON output file.
- For the generated tokens, use `tokenizer.decode` to convert them back to strings, and save them as a new dictionary item with “`model_response`” as the key. So the saved

JSON output should have the following keys in each entry:

```
{
  "instruction": "Name the author of 'Pride and Prejudice'.",
  "input": "",
  "output": "Jane Austen.",
  "model_response": "The author of 'Pride and Prejudice' is Jane Austen."
},
```

This is important because the later evaluation script relies on the exact keys.

Also, since we use 10% as test split, the output JSON should have 110 entries.

- You can call the generate function to the end of training in the main function. Or, you may write a standalone script for convenient use on any fine-tuned model. A “--generate_response” argument is prepared to control this action.
- After the responses on the test set are generated, use the provided ollama_evaluate_v2.py script to run an evaluation by:

```
$ python ollama_evaluate_v2.py \
  --file_path responses.json \
  --url 10.16.76.75 --port 8080
```

This script will send the judging task as http requests to an ollama-served gpt-oss (20B) model at the above IP address and port number.

If you are able to setup a gpt-oss (20B) model via ollama locally, you can also replace the IP to 127.0.0.1, and port number to 11434 (ollama default).

The server is running on a CPU-only desktop, so the evaluation speed would not be very fast (about 2~3 sec per sample). The time will be significantly longer if there are multiple requests at the same time.

(⚠ Don't abuse the server, otherwise your IP address will be banned)

The LLM-as-a-judge will return the evaluation results in average scores as follows:

```
Ollama server reachable at: http://10.16.76.75:8080
Scoring entries: 100%|
Number of scores: 110 of 110
Average score: 2.29
```

- In the report, include:
 - One plot of training and validation loss curves for a successful run of SFT.
 - Two screenshots of running the LLM-as-a-judge evaluation, for the non-masking and masking conditions (for instruction and input tokens), respectively.